

`</>` **main.py**
Code

`</>` **block.py**
Code

`</>` **game_engine.py**
Code

I'm attaching 3 files from a small Pygame game called Skyscraper Stack . It's a tower stacking game. A block slides left and right, you press Space or left click to drop it, and it's supposed to land on top of the previous block. Anything hanging over the edge gets trimmed off, so the tower gets narrower. If the block misses the tower completely, the tower collapses and you get a game over screen. It's Python 3.10+ with Pygame, running on Windows.

Right now the drop logic is backwards. If I land a block on the tower it's instantly game over (Height stays at 0), and if I drop it in empty air the game just keeps building. Valid overlaps should stack and complete misses should end the game.

There's also a second problem. The game over screen says "Press [Space] or [R] to Play Again" but Space does nothing there, only R and mouse click work. Make Space restart it too.

Please find the cause yourself, fix only what's needed, and don't change or restructure anything else. Give me back only the files you changed, as full file contents

`main.py` and `block.py` do not need changes. The two bugs are both in `game_engine.py` :

- Drop success condition is reversed: `overlap <= 0` should be `overlap > 0` .
- Game-over restart checks `R` but not `Space` .

`</>` Python

```
import random
import pygame
from game.block import Block

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.block_height = 28
```

```

self.base_width = 180

self.font_title = pygame.font.SysFont(None, 38)
self.font_hud = pygame.font.SysFont(None, 28)
self.font_big = pygame.font.SysFont(None, 46)

self.reset()

def get_color(self, index):
    palette = [

```

The overlap calculation itself was already correct; only its success test was backwards. The restart text was also already correct, so only the event condition needed changing. `</> game_engine`

Thank youuu. Next change, same rules: only touch what's needed and give me back just the changed files in full with their paths. Add a Perfect Placement bonus. Right now any drop that's slightly off center permanently shaves the block down. If a dropped block lines up almost exactly with the top of the tower (use a forgiving tolerance, around 12 pixels), don't trim it. Snap it onto the block below so it keeps the same width. Show a "PERFECT!" popup that fades out, and give some bonus score for it. If the player lands 2 or more perfect drops in a row, the placed block should get a little wider again, restoring some of the lost width, but never past the original width or off the screen edges. Make sure the restored width really ends up on the tower, not just on the moving block. A non-perfect drop breaks the streak. Reset all of this when the game restarts. Put the tolerance and bonus values as constants at the top so I can tweak them.

Worked for 13s

Only `game/game_engine.py` needs changing. `main.py` and `block.py` stay untouched. I've also preserved the two fixes from the previous change (correct overlap detection and Space-to-restart).

game/game_engine.py

`</> Python`

```

import random
import pygame
from game.block import Block

```

```

PERFECT_TOLERANCE = 12.0
PERFECT_BONUS = 3
PERFECT_RESTORE_WIDTH = 12.0
PERFECT_POPUP_DURATION = 800

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.block_height = 28
        self.base_width = 180

        self.font_title = pygame.font.SysFont(None, 38)
        self.font_hud = pygame.font.SysFont(None, 28)
        self.font_hig = pygame.font.SysFont(None, 46)

```

The key detail is that the restored width is applied directly to `new_block` **before it is appended to `self.stack`**, so the wider block is actually part of the permanent tower rather than merely affecting the moving block. `</>` `game_engine`

Good, that works. Next one, same rules: only touch what's needed and give me back just the changed files in full with their paths. Right now when a block gets trimmed, the cut off piece just vanishes. Add falling debris. The sliced off overhang should become its own piece, same color and same size as the part that was cut, that falls down with gravity while rotating, and gets removed once it's off screen. It should keep animating while the game continues, and shift down along with the tower when the camera scrolls. No debris on perfect drops, since nothing gets cut there. Clear it on restart. Put the Debris class in `game/block.py` so we don't add new files.

Worked for 7s

game/block.py

`</>` Python

```

import pygame

class Block:
    def __init__(self, x, y, width, height, color, speed=0):
        self.x = float(x)

```

```

        self.y = float(y)
        self.width = float(width)
        self.height = float(height)
        self.color = color
        self.speed = speed
        self.direction = 1

    @property
    def rect(self):
        return pygame.Rect(int(self.x), int(self.y), int(self.width), int(self.height))

    def update(self, screen_width):
        if self.speed == 0:
            return

```

game/game_engine.py

</> Python

```

import random
import pygame
from game.block import Block, Debris

PERFECT_TOLERANCE = 12.0
PERFECT_BONUS = 3
PERFECT_RESTORE_WIDTH = 12.0
PERFECT_POPUP_DURATION = 800

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.block_height = 28
        self.base_width = 180

        self.font_title = pygame.font.SysFont(None, 38)
        self.font_hud = pygame.font.SysFont(None, 28)
        self.font_big = pygame.font.SysFont(None, 46)

```

Good, that works. Last feature, same rules: only touch what's needed and give me back just the changed files in full with their paths. The background is always the same flat dark color no matter how high the tower gets. Make it change as the player climbs, with a smooth

vertical sky gradient that moves through atmospheric stages as the height goes up, for example evening blue, then dusk orange and pink, then deep purple night, then near black space with a few small stars. The first visible change should show up within the first few blocks, so it's easy to notice, and it should keep blending smoothly with no sudden jumps. Keep it cheap to draw, so don't loop over every pixel in Python every frame. The title, height and PERFECT! text have to stay readable on every stage, and the game over screen should still look right. Reset it on restart. Put the stage colors and how many blocks the full transition takes as constants at the top.

Worked for 6s

game/game_engine.py

</> Python

```
import random
import pygame
from game.block import Block, Debris

PERFECT_TOLERANCE = 12.0
PERFECT_BONUS = 3
PERFECT_RESTORE_WIDTH = 12.0
PERFECT_POPUP_DURATION = 800

BACKGROUND_STAGE_COLORS = [
    (45, 85, 145),    # Evening blue
    (220, 105, 75),   # Dusk orange
    (185, 85, 135),   # Dusk pink
    (65, 45, 105),    # Deep purple night
    (8, 10, 20),       # Near-black space
]
BACKGROUND_TRANSITION_BLOCKS = 40

class GameEngine:
```